

Implementación y Verificación de un Proceso ETL con SSIS

Etapas 3 — Tratamiento de Datos

Proyecto: Residuos Electrónicos y Consumo Tecnológico (RECT)

Jennifer Espitia, Richard Barajas y Cristian Leonardo Moscoso

Programa de Ingeniería de Sistemas y Computación, Universidad de Cundinamarca

Minería de Datos

Edison Cañón

29 de septiembre de 2026

Objetivo

Implementar y verificar un proceso ETL mediante SQL Server Integration Services (SSIS) para tratar los problemas de calidad identificados en la Etapa 2 del proyecto RECT, documentar los resultados de las iteraciones realizadas sobre el paquete y dejar registrada la evidencia disponible del repositorio y del avance de la aplicación Flask.

Diagnóstico inicial y reglas de tratamiento definidas

El diagnóstico de calidad de la Etapa 2 identificó, sobre los cuatro conjuntos de datos del proyecto (IDEAM, Indicadores globales, Encuesta Norte de Santander y DANE), los problemas resumidos en la siguiente tabla, base para las reglas de tratamiento definidas en SSIS:

Dataset	Registros	Variables	Compleitud	Duplicados
IDEAM (gestores)	377	35	94.66%	0 exactos / 50 por NIT
Indicadores globales	78,689	7	100%	2,505
Encuesta Norte de Santander	2,033	10	99.91%	116
DANE (nacional)	960	7	100%	0

Reglas de tratamiento definidas por problema

Campo afectado	Problema (Etapa 2)	Acción definida	Justificación
F1 (columna origen, encabezado de fuente)	Fila de encabezado ('Dominio') y filas vacías arrastradas por el origen Excel	Conditional Split: excluir filas donde F1 sea nulo o igual a "Dominio"	El origen Excel no reconoce la primera fila como encabezado real, por lo que esa fila y las vacías deben filtrarse antes de cargar
F1 (texto)	Espacios en blanco o cadenas vacías no capturados por ISNULL	Conditional Split: agregar TRIM(F1) != "" además de la validación de nulo	ISNULL no detecta una cadena vacía o con solo espacios; TRIM la normaliza para la comparación
F2, F3 (valores del indicador)	Registros con el indicador o su unidad sin diligenciar	Conditional Split: exigir !ISNULL(F2) && !ISNULL(F3)	Un registro sin el valor del indicador no aporta al análisis y debe separarse para revisión en vez de cargarse incompleto

Campo afectado	Problema (Etapa 2)	Acción definida	Justificación
Vigencia, col (IDEAM)	86.7% y 100% de nulos respectivamente (Etapa 2)	Excluir la columna 'col' por no tener ningún valor útil; conservar 'Vigencia' marcando el nulo explícitamente	Diagnosticado en la Etapa 2 como base para el tratamiento del dataset IDEAM
Registros duplicados (Indicadores globales, Encuesta NS)	2,505 y 116 duplicados exactos respectivamente (Etapa 2)	Componente Sort con la opción "eliminar filas con valores de ordenación duplicados"	Sort es el componente estándar de SSIS para deduplicar registros exactos
Departamento (DANE)	70 variantes de texto para ~35 entidades reales (Etapa 2)	Lookup contra una tabla de departamentos homologados	Un Lookup permite validar y corregir la categoría contra un catálogo de referencia

Diseño del Control Flow y del Data Flow

El paquete SSIS ('Package.dtsx', dentro del proyecto 'ETL_RECT_Filtrado0') contiene una tarea de flujo de datos (Data Flow Task) dentro del Flujo de control, encargada de aplicar las reglas de calidad definidas sobre el archivo de indicadores.

El Data Flow sigue esta estructura:

- Excel Source: lee el archivo de origen (indicadores relacionados con personas que utilizan celular, computador o red, según el nombre de usuario configurado en la conexión: RECT_PersonasQueUtilizanCelularComputadorORed).
- Conditional Split: aplica las reglas de calidad sobre las columnas F1, F2 y F3, dirigiendo las filas válidas a la salida "Datos_Limpios" y el resto a la salida por defecto ("Conditional Split Default Output").
- OLE DB Destination: inserta las filas de la salida seleccionada en la base de datos SQL Server (conexión localhost/master).

El archivo Excel de origen se conserva sin sobrescribir en todo el proceso, lo que preserva los datos originales frente a cualquier transformación aplicada por el Conditional Split.

Configuración de los componentes utilizados

Conditional Split

Este es, hasta la fecha, el único componente de transformación del paquete. Se muestra a continuación su configuración en las tres versiones (iteraciones) que se probaron.

Administrador de conexiones OLE DB Destination

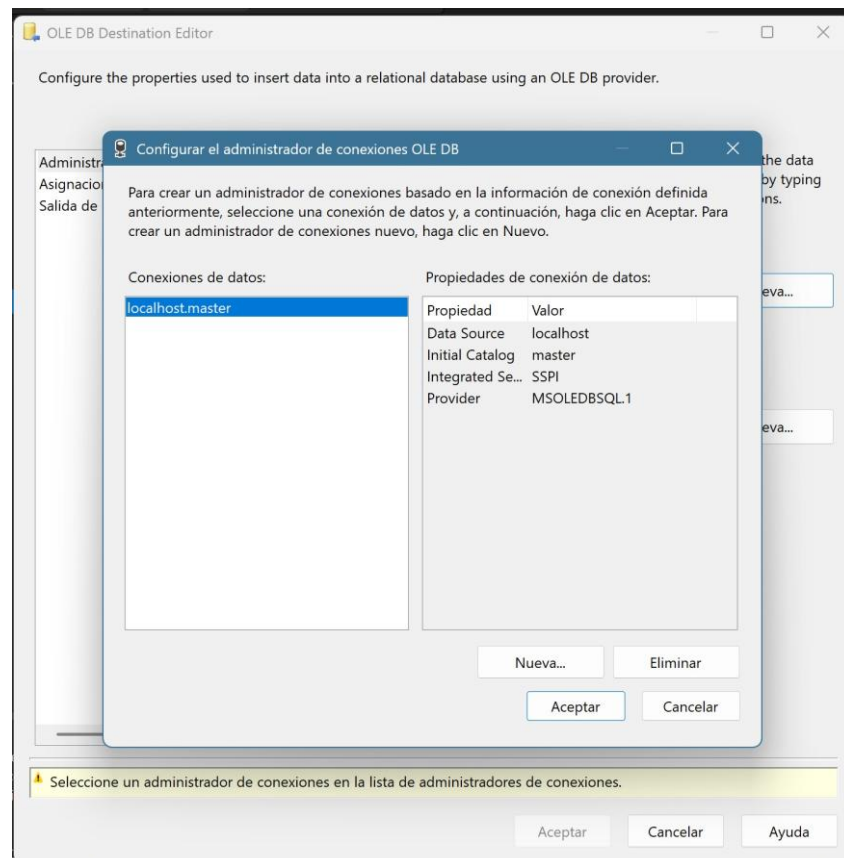


Figura 1. Cuadro de configuración del administrador de conexiones OLE DB del destino, apuntando al servidor local (Data Source: localhost, Initial Catalog: master, autenticación integrada de Windows).

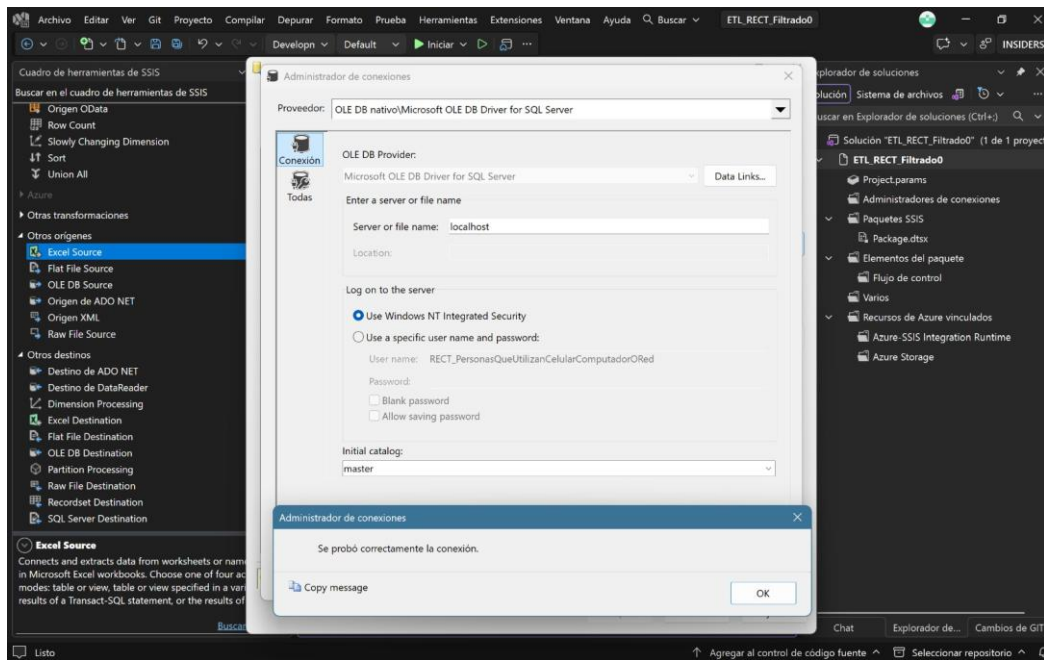


Figura 2. Prueba de conexión exitosa ("Se probó correctamente la conexión") desde el editor del Excel Source hacia el servidor SQL Server local. El nombre de usuario configurado, *RECT_PersonasQueUtilizanCelularComputadorORed*, identifica que este paquete procesa el indicador de personas que usan celular, computador o red.

Evidencias de ejecución de las iteraciones

Se identificaron tres versiones sucesivas de la condición del Conditional Split, cada una ejecutada sobre el Data Flow. A continuación se documenta cada iteración con su regla, su ajuste respecto a la anterior y el resultado real observado.

Iteración 1 — filtro base de encabezado y nulos

Regla aplicada: `!ISNULL(F1) && F1 != "Dominio"`. Esta primera versión solo descarta la fila de encabezado mal interpretada por el origen Excel ("Dominio") y las filas con F1 nulo.

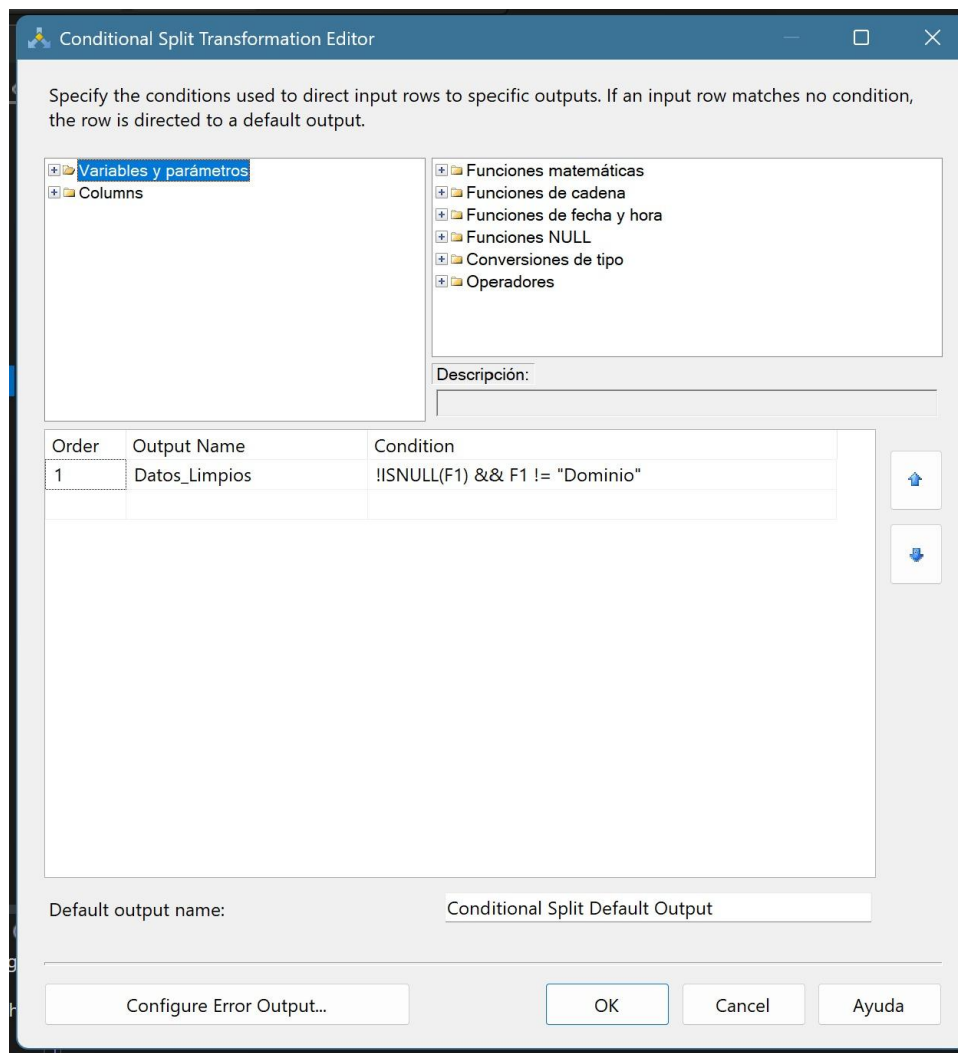


Figura 3. Editor del Conditional Split con la condición de la iteración 1: !ISNULL(F1) && F1 != "Dominio".

Resultado de ejecución (56 filas de entrada):

Figura 5. Segunda ejecución de la misma regla, esta vez con el OLE DB Destination conectado a la salida "Datos_Limpios": de las 56 filas de entrada, 39 pasaron el filtro y se cargaron a la base de datos; las 17 restantes correspondían a la fila de encabezado y a registros con F1 vacío.

Ajuste motivado por este resultado: al revisar las 17 filas descartadas (Figura 4) se detectó que algunas contenían valores en F1 con espacios en blanco que ISNULL no captura como nulos, lo que llevó a agregar una validación de cadena vacía (TRIM) en la siguiente iteración.

Iteración 2 — validación de formato de texto

Regla aplicada: !ISNULL(F1) && F1 != "Dominio" && !TOKENABAND..., donde TOKENABAND corresponde a una función de validación de formato/token sobre F1 agregada en esta iteración.

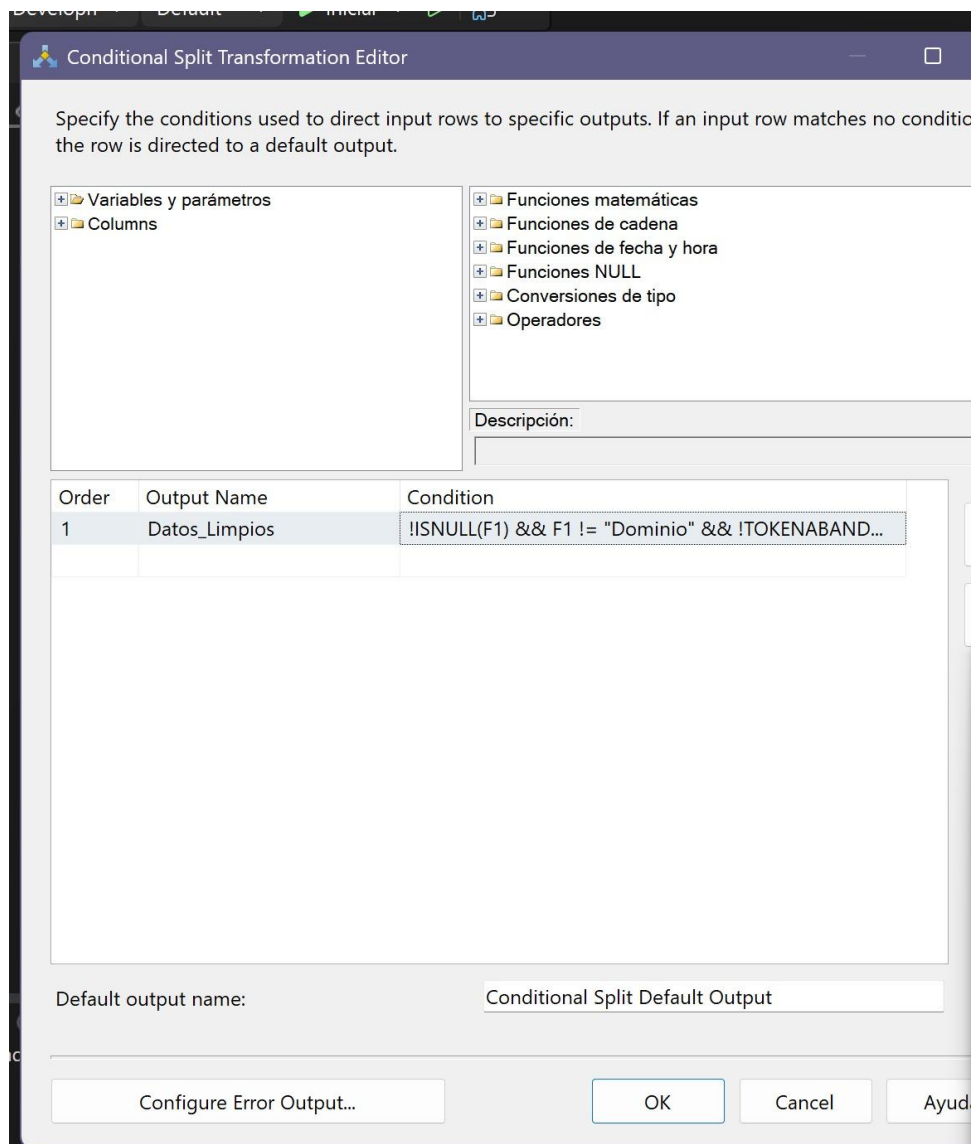


Figura 6. Editor del Conditional Split con la condición ampliada de la iteración 2, agregando una validación adicional de formato sobre F1.

Iteración 3 — validación completa de F1, F2 y F3

Regla aplicada: !ISNULL(F1) && TRIM(F1) != "" && F1 != "Dominio" && !ISNULL(F2) && !ISNULL(F3).

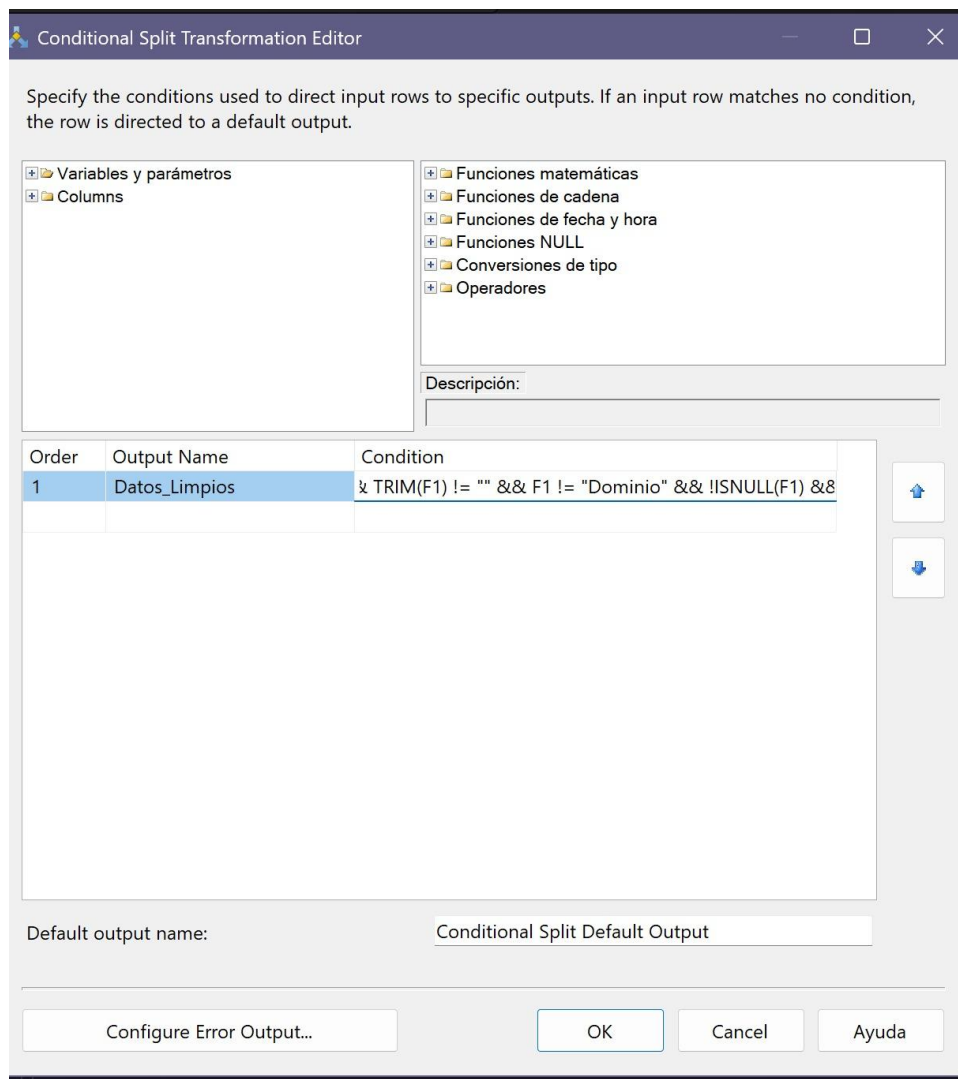


Figura 7. Editor del Conditional Split con la condición de la iteración 3, que añade $TRIM(F1) \neq ""$ y exige que $F2$ y $F3$ tampoco sean nulos.

Resultado de ejecución (128 filas de entrada, sobre una fuente distinta a la de la iteración 1 — se observan conexiones adicionales "Excel 1", "Excel 2" en el paquete):



Figura 8. Ejecución de la iteración 3: de 128 filas de entrada, únicamente 1 pasó el filtro hacia "Datos_Limpios".

Hallazgo y causa: la caída de 128 a 1 fila no se debe a un error del filtro sobre F1, sino a que la nueva exigencia `!ISNULL(F2) && !ISNULL(F3)` descartó prácticamente todo el archivo, lo que indica que las columnas F2 y/o F3 vienen vacías en la gran mayoría de los registros de esta fuente.

Vista general del paquete y conexiones configuradas

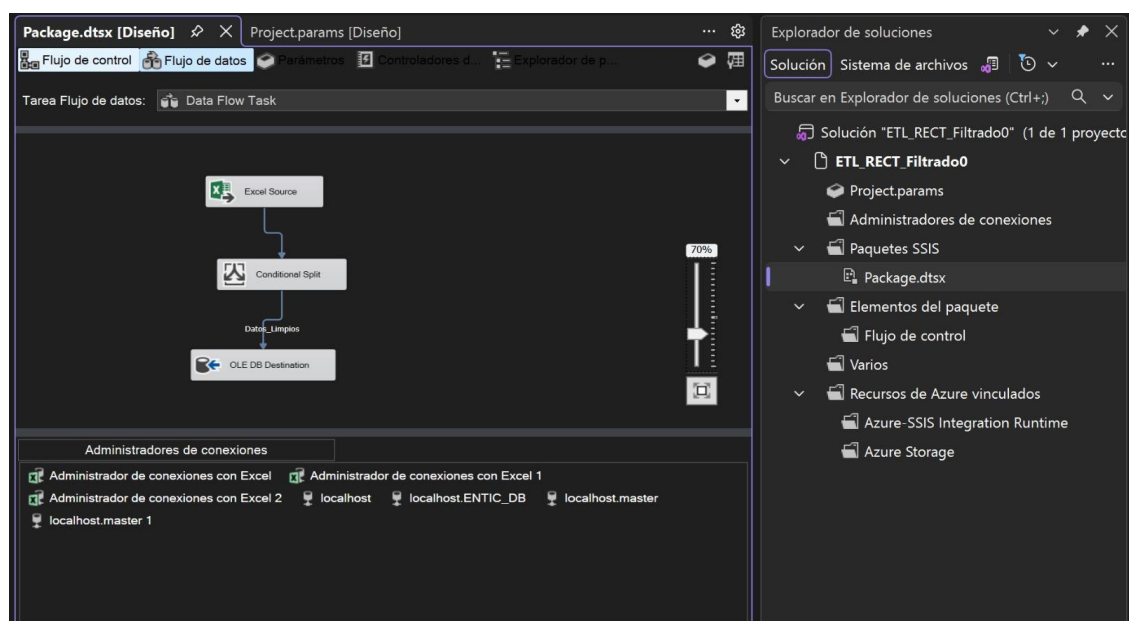


Figura 9. Explorador de soluciones del proyecto `ETL_RECT_Filtrado0`, mostrando el Data Flow (`Excel Source` → `Conditional Split` → `OLE DB Destination`) y la lista de administradores de conexiones configurados: `Excel`, `Excel 1`, `Excel 2`, `localhost`, `localhost.ENTIC_DB`, `localhost.master` y `localhost.master 1`. La existencia de conexiones separadas por fuente (`Excel 1`, `Excel 2`, `localhost.ENTIC_DB`) refleja que el tratamiento de los distintos datasets del proyecto (`IDEAM`, `ENTIC/DANE`) se aborda con conexiones dedicadas dentro del mismo proyecto.

Problemas encontrados durante la construcción del ETL

Durante el desarrollo se presentaron los siguientes problemas, reportados por el equipo:

Dataset / componente	Problema reportado	Causa señalada	Estado
IDEAM (gestores)	El filtro de calidad dejaba pasar muy pocos registros; la mayoría se descartaba por valores nulos	Coincide con el diagnóstico de la Etapa 2: los campos "Vigencia" (86.7% nulos) y "col" (100% nulos)	Identificado a partir del diagnóstico de la Etapa 2

Dataset / componente	Problema reportado	Causa señalada	Estado
		generan un volumen alto de filas incompletas	
IDEAM (gestores)	El paquete no permitía guardar los datos en la base de datos	Configuración del OLE DB Destination	Reportado por el equipo durante la construcción
ENTIC / DANE	Error al guardar en la base de datos	El administrador de conexiones apuntaba a "localhost" en general en vez de a la instancia/base de datos específica	Resuelto al crear la conexión dedicada "localhost.ENTIC_DB"

Manejo de errores y registros enviados a revisión

Los registros que no cumplen la condición del Conditional Split se dirigen a la salida "Conditional Split Default Output". En las pruebas documentadas (Figura 4) esta salida se conectó directamente al OLE DB Destination para inspeccionar qué filas rechazaba el filtro, lo que permitió identificar los ajustes aplicados entre iteraciones.

Comparación de resultados

La siguiente tabla resume los conteos observados en las ejecuciones documentadas:

Iteración	Regla añadida respecto a la anterior	Filas recibidas	Filas aceptadas (Datos_Limpios)	Filas rechazadas
1	Excluir F1 nulo y F1 = "Dominio"	56	39	17
2	+ validación de formato/token sobre F1	—	—	—
3	+ TRIM(F1) != "" y F2, F3 no nulos	128	1	127

Lectura de los indicadores: la iteración 1 acepta el 69.6% de los registros de su archivo de origen (39/56), descartando principalmente la fila de encabezado y registros con F1 vacío. La iteración 3, sobre una fuente distinta y de mayor tamaño (128 filas), acepta apenas el 0.8% (1/128) al exigir adicionalmente que F2 y F3 no sean nulos — una caída que, como se explicó, refleja que esas dos columnas vienen vacías en casi todo ese archivo y no un error de la regla en sí.

Gestión en GitHub

El repositorio del proyecto (Jennespi/RECT) registra el avance de esta etapa en la rama feature/etapa-3, integrada a main mediante un pull request.

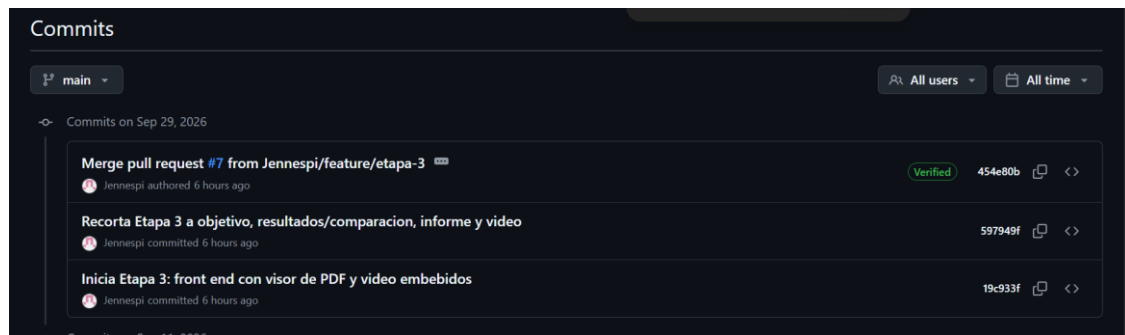


Figura 10. Historial de commits: "Inicia Etapa 3: front end con visor de PDF y video embebidos", "Recorta Etapa 3 a objetivo, resultados/comparación, informe y video", y el merge "Merge pull request #7 from Jennespi/feature/etapa-3" (verificado por GitHub).

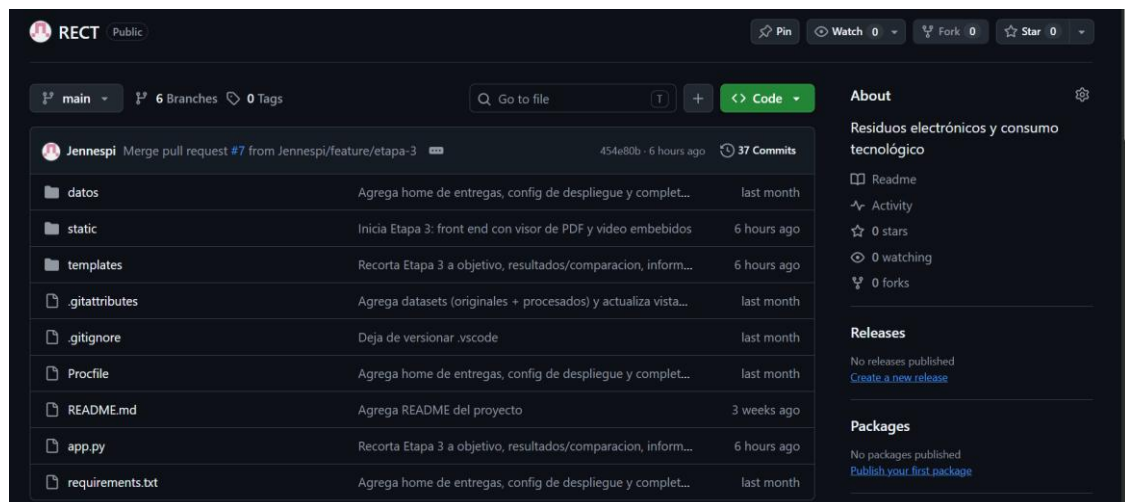


Figura 11. Estructura del repositorio en main tras el merge del PR #7: las carpetas "datos", "static" y "templates" ya reflejan el trabajo de la Etapa 3 (front end con visor de PDF y video).

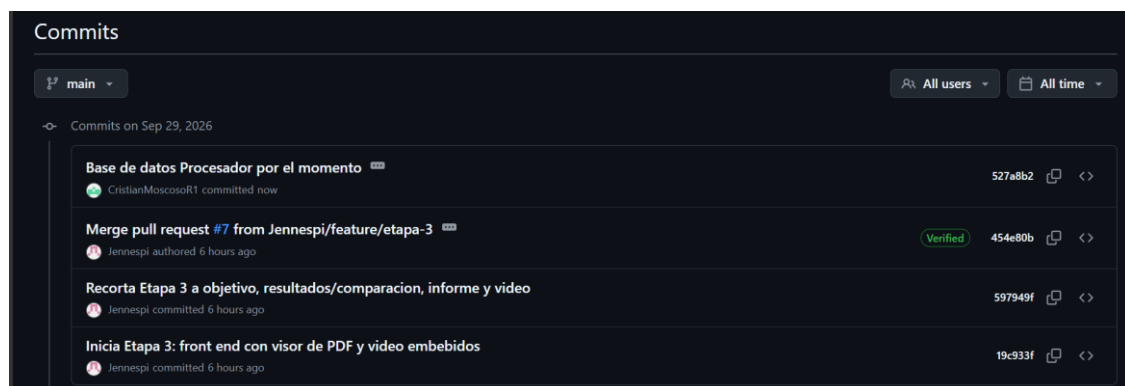


Figura 12. Historial actualizado con un commit posterior de otro integrante del equipo (CristianMoscosoR1): "Base de datos Procesador por el momento", que refleja el trabajo del equipo sobre la base de datos del ETL.

Enlaces del proyecto

Repositorio público en GitHub: <https://github.com/Jennespi/RECT>

Rama de esta entrega (feature/etapa-3):

<https://github.com/Jennespi/RECT/tree/feature/etapa-3>

Pull request de esta etapa (#7): <https://github.com/Jennespi/RECT/pull/7>

Aplicación publicada: <https://rect-tfdr.onrender.com>

Video de demostración: <https://www.youtube.com/watch?v=SM41JHEF8cE>